

TFI – Programacion II

Alumnos: Marcos Rios (Comision 9)
Elias Pfaeffli (Comision 8)

Tutor/a: Flor Gubiotti (Comision 9)
Jerónimo Cortez (Comision 8)

Profesor: Cinthia Rigoni (Comision 9)
Ariel Enferrel (Comision 8)

Fecha: 20/11/2025

Materia: Programacion

Tema: TFI - Sistema Empresa-DomicilioFiscal

1. Links:

Repositorio GitHub: https://github.com/MarcosRios547/TFI_Empresa-DomicilioFiscal.git

Video: <https://youtu.be/4nhYdJoMA90>

En caso de que no funcione el link anterior se puede visualizar el video en este:

<https://drive.google.com/file/d/1SoqXblZ-2Q7wUAqHpcqkDoyuOujmKTsb/view?usp=sharing>

2. ELECCIÓN DEL DOMINIO

Dominio: Empresa → DomicilioFiscal

Justificación: Elegimos este dominio porque representa un caso real del mundo empresarial donde cada empresa debe tener un único domicilio fiscal registrado. La relación 1→1 es natural: cada empresa tiene exactamente un domicilio fiscal y este no puede ser compartido. La unidireccionalidad tiene sentido desde el modelo de objetos: la empresa conoce su domicilio, pero el domicilio no necesita conocer a qué empresa pertenece.

3. DISEÑO DEL SISTEMA

3.1 Diagrama UML

El diagrama completo se encuentra en docs/UML_Diagrama.png y muestra:

- Clases **Empresa** y **DomicilioFiscal** con todos sus atributos
- Relación 1→1 unidireccional: Empresa tiene private DomicilioFiscal domicilioFiscal
- Interfaz **GenericDao<T>** implementada por EmpresaDao y DomicilioFiscalDao
- Servicios que orquestan transacciones
- AppMenu que consume los servicios

3.2 Decisión Clave: Implementación 1→1 en BD

Utilizamos **Foreign Key ÚNICA** en la tabla empresa:

```
CREATE TABLE empresa (  
    domicilio_id BIGINT NOT NULL UNIQUE,  
    FOREIGN KEY (domicilio_id) REFERENCES domicilio_fiscal(id)  
    ON DELETE CASCADE  
);
```

La restricción UNIQUE en domicilio_id garantiza que un domicilio solo puede ser referenciado por una empresa, implementando así la relación 1→1 a nivel de base de datos.

Baja lógica: Implementamos el campo eliminado en ambas tablas para marcar registros sin borrarlos físicamente, manteniendo histórico y evitando problemas de integridad referencial.

4. ARQUITECTURA POR CAPAS

Capa CONFIG

DatabaseConnection.java - Gestión centralizada de conexiones MySQL mediante patrón Singleton.

Capa ENTITIES

Empresa.java y **DomicilioFiscal.java** - POJOs con constructores, getters/setters y toString(). La relación 1→1 se modela con private DomicilioFiscal domicilioFiscal en Empresa.

Capa DAO

GenericDao<T> - Interfaz con operaciones CRUD básicas. Característica crítica: todos los métodos reciben Connection conn como parámetro, permitiendo que múltiples operaciones compartan la misma transacción.

EmpresaDao y **DomicilioFiscalDao** - Implementan GenericDao usando PreparedStatement en todas las operaciones para prevenir SQL injection. Incluyen métodos adicionales: leerPorCuit() y leerPorRazonSocial().

Orden de operaciones en crear():

```
// 1. Crear domicilio primero
domicilioDao.crear(e.getDomicilioFiscal(), conn);
// 2. Crear empresa referenciando el domicilio_id
empresaDao.crear(e, conn);
```

Capa SERVICE

EmpresaServicio y **DomicilioFiscalServicio** - Manejan lógica de negocio y transacciones.

Patrón transaccional implementado:

```
Connection conn = null;
try {
    conn = DatabaseConnection.getConnection();
    conn.setAutoCommit(false); // Iniciar transacción

    // Validaciones de negocio
    Empresa existente = empresaDao.leerPorCuit(e.getCuit(), conn);
```

```

    if (existente != null) return false;

    // Operaciones
    empresaDao.crear(e, conn);

    conn.commit(); // Confirmar si todo OK
    return true;
} catch (SQLException ex) {
    if (conn != null) conn.rollback(); // Revertir si error
    return false;
} finally {
    if (conn != null) {
        conn.setAutoCommit(true);
        conn.close();
    }
}
}

```

Capa UTIL

InputValidator.java - Validaciones básicas de formato (CUIT, campos obligatorios, entradas numéricas).

Capa MAIN

Main.java - Punto de entrada.

AppMenu.java - Menú interactivo con manejo robusto de errores (método leerIntSafe() captura excepciones sin crashear), validaciones tempranas con InputValidator, y confirmación antes de eliminar.

5. PERSISTENCIA Y BASE DE DATOS

Estructura de Tablas

domicilio_fiscal:

- PK: id (BIGINT AUTO_INCREMENT)
- Campos: eliminado, calle, numero, ciudad, provincia, codigo_postal, pais
- Índices: eliminado, ciudad

empresa:

- PK: id (BIGINT AUTO_INCREMENT)
- UK: cuit, domicilio_id (UNIQUE garantiza 1→1)
- FK: domicilio_id → domicilio_fiscal(id) ON DELETE CASCADE
- Índices: cuit, eliminado, razon_social

Motor: InnoDB (soporta transacciones ACID y claves foráneas)

Charset: utf8mb4 (soporta caracteres especiales)

Transacciones y Consistencia

Las operaciones compuestas (crear empresa + domicilio) se ejecutan en una transacción atómica. Si falla cualquier paso, el rollback deshace todas las operaciones, garantizando que la base de datos nunca quede en estado inconsistente.

6. VALIDACIONES Y REGLAS DE NEGOCIO

Validaciones por Capa

En AppMenu (InputValidator):

- CUIT con formato razonable (10-13 dígitos)
- Campos obligatorios no vacíos
- Entradas numéricas válidas

En Service:

- **RN-01: CUIT Único** - Verificado antes de crear y actualizar
- **RN-02: Domicilio Obligatorio** - Toda empresa debe tener domicilio fiscal
- **RN-03: Baja Lógica** - Registros marcados como eliminado=TRUE, no borrados
- **RN-04: Integridad Referencial** - FK con CASCADE garantiza consistencia

Validación especial en actualizar():

```
// Si cambió el CUIT, verificar que no esté en uso por otra empresa
if (!e.getCuit().equalsIgnoreCase(exists.getCuit())) {
    Empresa conflicto = empresaDao.leerPorCuit(e.getCuit(), conn);
    if (conflicto != null && !conflicto.getId().equals(e.getId())) {
        return false; // Conflicto detectado
    }
}
```

7. PRUEBAS REALIZADAS

Pruebas Funcionales Exitosas

Test 1: Crear Empresa Completa

- Entrada: Tech Solutions SA, CUIT 30-71234567-8, domicilio completo
- Resultado: ✓ Empresa creada con ID auto-generado, ambos registros en BD

Test 2: CUIT Duplicado (Rollback)

- Entrada: Intentar crear segunda empresa con CUIT 30-71234567-8
- Resultado: ✓ Mensaje "Ya existe empresa con ese CUIT", rollback ejecutado, BD intacta

Test 3: Búsqueda por CUIT

- Entrada: CUIT 30-71234567-8
- Resultado: ✓ Empresa encontrada con su domicilio completo

Test 4: Búsqueda por Razón Social

- Entrada: Texto "tech"
- SQL: WHERE LOWER(razon_social) LIKE '%tech%'
- Resultado: ✓ Encontró "Tech Solutions SA" (case-insensitive)

Test 5: Actualización Transaccional

- Entrada: Cambiar email y número de domicilio
- Resultado: ✓ Ambas actualizaciones en una transacción

Test 6: Eliminación Lógica

- Entrada: Eliminar empresa ID 1
- Verificación SQL: SELECT * FROM empresa WHERE id=1 → eliminado=TRUE
- Resultado: ✓ Registro marcado, no aparece en listados

Test 7: Manejo de Errores

- Entrada: Ingresar "abc" en menú numérico
- Resultado: ✓ leerIntSafe() captura excepción, no crashea

Consultas SQL de Verificación

Ver empresas con domicilios:

```
SELECT e.razon_social, e.cuit, d.calle, d.ciudad
FROM empresa e
JOIN domicilio_fiscal d ON e.domicilio_id = d.id
WHERE e.eliminado = FALSE;
```

Verificar unicidad 1→1:

```
SELECT domicilio_id, COUNT(*)
FROM empresa
GROUP BY domicilio_id
HAVING COUNT(*) > 1;
```

-- Resultado: 0 filas (correcto)

8. CONCLUSIONES Y MEJORAS FUTURAS

Objetivos Cumplidos

- ✓ Relación 1→1 unidireccional en código (Empresa → DomicilioFiscal) y BD (FK UNIQUE)
- ✓ Patrón DAO con GenericDao<T> y PreparedStatement en todas las operaciones
- ✓ Transacciones con commit/rollback en todas las modificaciones
- ✓ Baja lógica con campo eliminado
- ✓ CRUD completo con búsquedas por ID, CUIT y razón social
- ✓ Validaciones en múltiples capas
- ✓ Manejo robusto de errores sin crasheos

Aprendizajes Clave

- **Transacciones ACID:** Comprender cómo setAutoCommit(false), commit() y rollback() mantienen consistencia fue fundamental
- **Patrón DAO:** Separar lógica de acceso a datos facilitó testing y mantenimiento
- **Validaciones en capas:** Validar en menú + service + BD proporciona defensa en profundidad
- **Trabajo en equipo:** División por capas permitió trabajo paralelo eficiente

Mejoras Futuras

Corto plazo:

- Validación estricta de CUIT con dígito verificador AFIP
- Pool de conexiones (HikariCP)
- Tests unitarios con JUnit 5 + Mockito

Mediano plazo:

- Paginación en listados
- Exportación de reportes (PDF con iText)
- Logging con Log4j

Largo plazo:

- Migración a Spring Boot + Spring Data JPA
- API REST con endpoints JSON
- Frontend web (React/Angular)
- Microservicios

Reflexión Final

Este proyecto consolidó nuestra comprensión de persistencia con JDBC, arquitectura en capas y transacciones.